

Inigo Quilez :: articles :: box functions



Intro

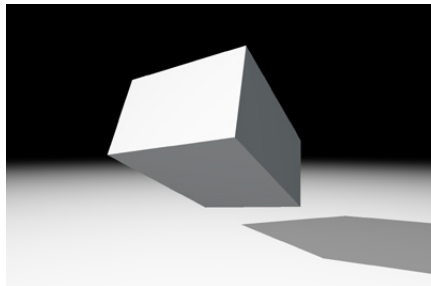
A sphere are simple geometry shape described by a simple equation and therefore it accepts **analytic solutions** to many problems involving projections, occlusion, shadows, motion blur, etc. Similary, boxes are simple shapes that also accept closed form expresions for many such problems. This page collects some of those that I have collected (more often than not, derived myself).

Functions

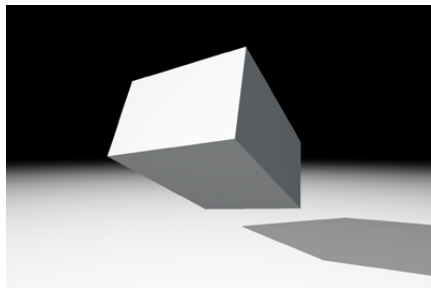
Notes on helper functions:

* maxcomp() computes the largest of the coordinates of a vector.

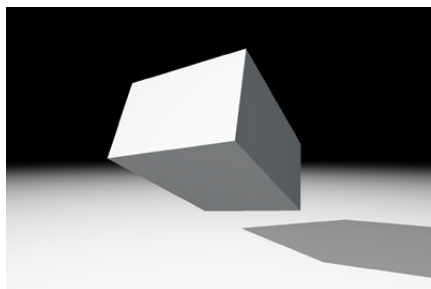
* msign() for each component in a vector, returns -1 if it's negative or zero, and +1 otherwise

**Box Distance**
[Example / Code](#)
[Tutorial](#)

```
// Calcs signed distance to a box of semi-size "rad" centered at
// the origin. It works in 2D, 3D, 4D any number of dimensions by
// replacing the vec data types.
float boxDistance( in vec3 p, in vec3 rad )
{
    vec2 d = abs(p)-rad;
    return length(max(d,0.0)) + min(maxcomp(d),0.0);
}
```

**Box Gradient / Normal**
[Example / Code](#)

```
// Calcs the gradient of the distance to a box. Works at
// any point in space, including the interior of the box.
float boxGradient( in vec3 p, in vec3 rad )
{
    vec3 d = abs(p)-rad;
    vec3 s = msign(p);
    float g = maxcomp(d);
    return s*((g>0.0) ? normalize(max(d,0.0)) :
                step(d.yzx,d.xyz)*step(d.zxy,d.yz));
}
```

**Box Intersection**
[Example / Code](#)

```
// Calcs intersection and exit distances, and normal at intersection.
// The ray must be in box/object space. If you have multiple boxes all
// aligned to the same axis, you can precompute 1/rd. If you have
// multiple boxes but they are not alligned to each other, use the
// "Generic" box intersector bellow this one.
vec2 boxIntersection( in vec3 ro, in vec3 rd, in vec3 rad, out vec3 on )
{

```

```

    vec3 m = 1.0/rd;
    vec3 n = m*ro;
    vec3 k = abs(m)*rad;
    vec3 t1 = -n - k;
    vec3 t2 = -n + k;

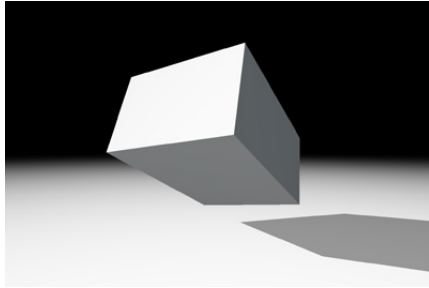
    float tN = max( max( t1.x, t1.y ), t1.z );
    float tF = min( min( t2.x, t2.y ), t2.z );

    if( tN>tF || tF<0.0) return vec2(-1.0); // no intersection

    oN = -sign(rd)*step(t1.yzx,t1.xyz)*step(t1.zxy,t1.xyz);

    return vec2( tN, tF );
}

```



Box Intersection Generic

[Example / Code](#)

```

// Calcs intersection and exit distances, normal, face and UVs
// row is the ray origin in world space
// rdw is the ray direction in world space
// txx is the world-to-box transformation
// txi is the box-to-world transformation
// ro and rd are in world space
// rad is the half-length of the box
//
// oT contains the entry and exit points
// oN is the normal in world space
// oU contains the UVs at the intersection point
// oF contains the index if the intersected face [0..5]
bool boxIntersect( in vec3 row, in vec3 rdw, in mat4 txx, in mat4 txi, in vec
                  out vec2 oT, out vec3 oN, out vec2 oU, out int oF )
{
    // convert from world to box space
    vec3 rd = (txx*vec4(rdw,0.0)).xyz;
    vec3 ro = (txx*vec4(row,1.0)).xyz;

    // ray-box intersection in box space
    vec3 m = 1.0/rd;
    vec3 s = vec3((rd.x<0.0)?1.0:-1.0,
                  (rd.y<0.0)?1.0:-1.0,
                  (rd.z<0.0)?1.0:-1.0);
    vec3 t1 = m*(-ro + s*rad);
    vec3 t2 = m*(-ro - s*rad);

    float tN = max( max( t1.x, t1.y ), t1.z );
    float tF = min( min( t2.x, t2.y ), t2.z );

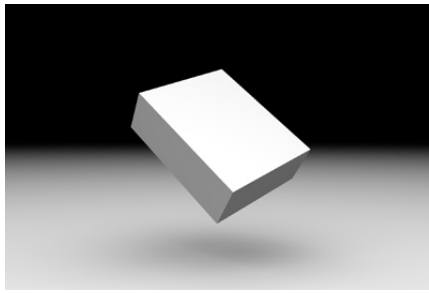
    if( tN>tF || tF<0.0) return false;

    // compute normal (in world space), face and UV
    if( t1.x>t1.y && t1.x>t1.z ) { oN=txi[0].xyz*s.x; oU=ro.yz+rd.yz*t1.x; oF
    else if( t1.y>t1.z ) { oN=txi[1].xyz*s.y; oU=ro.zx+rd.zx*t1.y; oF
    else { oN=txi[2].xyz*s.z; oU=ro.xy+rd.xy*t1.z; oF

    oT = vec2(tN,tF);

    return true;
}

```



Box Ambient Occlusion

[Maths / Article](#)

```
float boxOcclusion( in vec3 pos, in vec3 nor,
                  in mat4 txx, // box rotation+position
                  in vec3 rad ) // box size
{
    vec3 p = (txx*vec4(pos,1.0)).xyz;
    vec3 n = (txx*vec4(nor,0.0)).xyz;

    // 8 verts
    vec3 v0 = normalize( vec3(-1.0,-1.0,-1.0)*rad - p);
    vec3 v1 = normalize( vec3( 1.0,-1.0,-1.0)*rad - p);
    vec3 v2 = normalize( vec3(-1.0, 1.0,-1.0)*rad - p);
    vec3 v3 = normalize( vec3( 1.0, 1.0,-1.0)*rad - p);
    vec3 v4 = normalize( vec3(-1.0,-1.0, 1.0)*rad - p);
    vec3 v5 = normalize( vec3( 1.0,-1.0, 1.0)*rad - p);
    vec3 v6 = normalize( vec3(-1.0, 1.0, 1.0)*rad - p);
    vec3 v7 = normalize( vec3( 1.0, 1.0, 1.0)*rad - p);

    // 12 edges
    float k02 = dot(n,normalize(cross(v2,v0)))*acos(dot(v0,v2));
    float k23 = dot(n,normalize(cross(v3,v2)))*acos(dot(v2,v3));
    float k31 = dot(n,normalize(cross(v1,v3)))*acos(dot(v3,v1));
    float k10 = dot(n,normalize(cross(v0,v1)))*acos(dot(v1,v0));
    float k45 = dot(n,normalize(cross(v5,v4)))*acos(dot(v4,v5));
    float k57 = dot(n,normalize(cross(v7,v5)))*acos(dot(v5,v7));
    float k76 = dot(n,normalize(cross(v6,v7)))*acos(dot(v7,v6));
    float k37 = dot(n,normalize(cross(v7,v3)))*acos(dot(v3,v7));
    float k64 = dot(n,normalize(cross(v4,v6)))*acos(dot(v6,v4));
    float k51 = dot(n,normalize(cross(v1,v5)))*acos(dot(v5,v1));
    float k04 = dot(n,normalize(cross(v4,v0)))*acos(dot(v0,v4));
    float k62 = dot(n,normalize(cross(v2,v6)))*acos(dot(v6,v2));

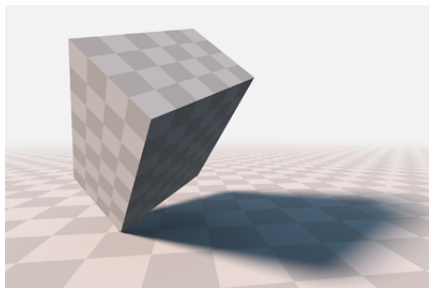
    // 6 faces
    float occ = 0.0;
    occ += ( k02 + k23 + k31 + k10 ) * step( 0.0, v0.z );
    occ += ( k45 + k57 + k76 + k64 ) * step( 0.0, -v4.z );
    occ += ( k51 - k31 + k37 - k57 ) * step( 0.0, -v5.x );
    occ += ( k04 - k64 + k62 - k02 ) * step( 0.0, v0.x );
    occ += (-k76 - k37 - k23 - k62) * step( 0.0, -v6.y );
    occ += (-k10 - k51 - k45 - k04) * step( 0.0, v0.y );

    return occ / 6.283185;
}
```

This code will work only when the box is fully visible from the shading point.

For boxes that are partially below the visibility horizon, use this code that does

clipping in the polygons instead: <https://www.shadertoy.com/view/4sSXDV>



Box Soft Shadows

[Example / Code](#)

```
float boxSoftShadow( in vec3 row, in vec3 rdw,
                   in mat4 txx, // box rotation+position
                   in vec3 rad, // box semi-size
                   in float sk ) // shadow softness (try 8.0)
{
    vec3 rd = (txx*vec4(rdw,0.0)).xyz;
    vec3 ro = (txx*vec4(row,1.0)).xyz;

    vec3 m = 1.0/rd;
    vec3 n = m*ro;
    vec3 k = abs(m)*rad;
    vec3 t1 = -n - k;
    vec3 t2 = -n + k;

    float tN = max( max( t1.x, t1.y ), t1.z );
    float tF = min( min( t2.x, t2.y ), t2.z );

    if( tN>tF || tF<0.0)
```

```

{
    float sh = 1.0;
    sh = segShadow( ro.xyz, rd.xyz, rad.xyz, sh );
    sh = segShadow( ro.yzx, rd.yzx, rad.yzx, sh );
    sh = segShadow( ro.zxy, rd.zxy, rad.zxy, sh );
    sh = clamp(sk*sqrt(sh),0.0,1.0);
    return sh*sh*(3.0-2.0*sh);
}
return 0.0;
}

float length2( in vec3 v ) { return dot(v,v); }
float segShadow( in vec3 ro, in vec3 rd, in vec3 pa, float sh )
{
    float k1 = 1.0 - rd.x*rd.x; // k = dot(rd.yz,rd.yz);
    float k4 = (ro.x-pa.x)*k1;
    float k6 = (ro.x+pa.x)*k1;
    vec2 k5 = ro.yz*k1;
    vec2 k7 = pa.yz*k1;
    float k2 = -dot(ro.yz,rd.yz);
    vec2 k3 = pa.yz*rd.yz;

    for( int i=0; i<4; i++ )
    {
        vec2 ss = vec2( i&1, i>>1)*2.0-1.0;
        float thx = k2 + dot(ss,k3);
        if( thx<0.0 ) continue; // behind
        float thy = clamp( -rd.x*thx, k4, k6 );
        sh = min( sh, length2( vec3(thy,k5-k7*ss) + rd*thx )/(thx*thx) );
    }
    return sh;
}

```

This function will produce super fast soft shadows that, while not accurate, are plausible.

The shadows will be sharper near the box and soften further away. You can control the softness of the shadow with the parameter **sk**: <https://www.shadertoy.com/view/4sSXDV>



Box Density

Example / Code

```

float boxDensity( vec3 wro, vec3 wrd,          // ray origin and direction
                  mat4 txx, vec3 rad, // box rot+pos, and size
                  float dbuffer )
{
    vec3 d = (txx*vec4(wrd,0.0)).xyz;
    vec3 o = (txx*vec4(wro,1.0)).xyz;

    // ray-box intersection in box space
    vec3 m = 1.0/d;
    vec3 n = m*o;
    vec3 k = abs(m)*rad;
    vec3 ta = -n - k;
    vec3 tb = -n + k;
    float tN = max( max( ta.x, ta.y ), ta.z );
    float tF = min( min( tb.x, tb.y ), tb.z );
    if( tN>tF || tF<0.0 ) return 0.0;

    // not visible (behind camera or behind dbuffer)
    if( tF<0.0 || tN>dbuffer ) return 0.0;

    // clip integration segment from camera to dbuffer
    tN = max( tN, 0.0 );
    tF = min( tF, dbuffer );

    // move ray to the intersection point
    o += tN*d; tF=tF-tN; tN=0.0;

    // density d(x,y,z) = [1-(x/rx)^2]*[1-(y/ry)^2]*[1-(z/rz)^2]
    // is analytically integrable:

    float ir2 = 1.0/(rad*rad);
    vec3 a = 1.0 - (o*o)*ir2;

```

```

vec3 b =      - 2.0*(o*d)*ir2;
vec3 c =      -      (d*d)*ir2;

float t1 = tF;
float t2 = t1*t1;
float t3 = t2*t1;
float t4 = t2*t2;
float t5 = t2*t3;
float t6 = t3*t3;
float t7 = t3*t4;

return (t1/1.0)*(a.x*a.y*a.z) +
      (t2/2.0)*(a.x*a.y*b.z + a.x*b.y*a.z + b.x*a.y*a.z) +
      (t3/3.0)*(a.x*a.y*c.z + a.x*b.y*b.z + a.x*c.y*a.z +
      b.x*a.y*b.z + b.x*b.y*a.z + c.x*a.y*a.z) +
      (t4/4.0)*(a.x*b.y*c.z + a.x*c.y*b.z + b.x*a.y*c.z +
      b.x*b.y*b.z + b.x*c.y*a.z + c.x*a.y*b.z +
      c.x*b.y*a.z) +
      (t5/5.0)*(a.x*c.y*c.z + b.x*b.y*c.z + b.x*c.y*b.z +
      c.x*a.y*c.z + c.x*b.y*b.z + c.x*c.y*a.z) +
      (t6/6.0)*(b.x*c.y*c.z + c.x*b.y*c.z + c.x*c.y*b.z) +
      (t7/7.0)*(c.x*c.y*c.z);
}

```

inigo quilez - learning computer graphics since 1994